AULA 2 · PRÁTICA

4 práticas de testes unitários em Python

pytest · fixtures · mocks · Tkinter
Pacote: praticas_testes_python.zip
(roteiro completo no README.md)

SESI
SENAI

Setup: preparar o ambiente

```
python --version          # 3.10 ou superior
python -m venv .venv
.venv\Scripts\activate    # Linux/Mac: source
.venv/bin/activate
pip install -r requirements.txt
pytest --version

cd pratica1_boletim
pytest -v
```

Lendo a saída

- . passou
- F falhou (veja a linha com >)
- s pulado (skip)
- pytest -x: para no 1º erro
- pytest -k nome: filtra por nome

Prática 1 · Boletim: o primeiro teste

Você vai aprender

- AAA: preparar, agir, conferir
- approx para decimais
- raises para erros esperados
- parametrize para várias bordas
- Esperado: 14 passed

```
Digitar Código do teste (com Professor)
```

Prática 2 · Senha: caça ao bug

Missão

- Rodar: 1 failed, 10 passed
- Achar o defeito na borda de 8 caracteres
- Registrar o relatório de defeito
- Corrigir e rodar de novo (regressão)
- Gabarito na pasta gabarito/

Código já pronto mas contém um BUG. Sua tarefa é rodar o teste, pesquisar e corrigir o erro (no arquivo: validador_senha.py). Também olhe o arquivo: (Relatorio_de_Defeito_Resolvido_Pratica2.docx) que contém um relatório pronto (apenas para ambientação).

Prática 3 · Conta bancária: fixtures

Você vai aprender

- fixture: cada teste recebe um objeto novo
- Exceção própria: SaldoInsuficienteError
- Efeito colateral: a falha não pode alterar o saldo
- property sem setter (encapsulamento)
- Esperado: 11 passed

```
Rodar testes (Já prontos)
```

Prática 4 · Conversor: testando desktop

3 níveis de teste

- A: lógica pura (conversor.py, sem tkinter)
- B: interface com Mock, sem abrir janela
- C: janela real com invoke() (só 1 teste)
- Regra de ouro: lógica fora da interface

```
Digitar Código do teste (seguir imagem).
Após testar. É esperado:

“========== 14 passed in 0.18s ==========”
```

Relatórios: defeito e teste

Relatório de defeito

- ID: BUG-001 · senha de 8 caracteres recusada
- Passos: validar_senha("Abcdef1!")
- Esperado: [ ] · Obtido: ["Use no mínimo 8…"]
- Severidade: alta · Prioridade: alta
- Status: aberto → corrigido → fechado

Relatório de teste (Prática 2)

- 11 executados · 10 aprovados · 1 reprovado
- Taxa = 10 ÷ 11 × 100 = 90,9%
- Defeito de severidade alta aberto
- Decisão: corrigir antes de liberar
- Após a correção: 11 de 11 = 100%

Cola do pytest

Recurso / comando	Para quê
pytest -v	um teste por linha, com o resultado
pytest -x · -k nome · -m "not gui"	para no 1º erro · filtra por nome · pula GUI real
assert a == pytest.approx(b)	comparar decimais
with pytest.raises(Erro, match="…")	exigir uma exceção
@pytest.mark.parametrize	várias entradas em um teste
@pytest.fixture	preparar objeto novo para cada teste
Mock() · patch("modulo.objeto")	dublê que registra chamadas · troca temporária
widget.invoke()	clicar num botão Tkinter sem mainloop